PatchLang v0.2.5

Language Specification & Project Architecture

2026-03-26

Design decisions ratified through structured Socratic debate and cross-agent consensus. This document is the authoritative reference for all PatchLang v0.2.0 changes.

Contents

Revision History

Answers to Reid's Questions

PatchLang Specification

- Design Principles
- File Conventions
- How This Document Is Organized

Examples

- Single-File Project
- Multi-File Project
- Ring Network (OptoCore)
- Channel Auto-Assignment with `[auto]`
- DRC Suppression with `@suppress`
- Split Directional Ports (Dante Template)
- Slot / Card Installation
- Internal Routes
- Config Block (Channel Labels)
- Template Composition

Project Structure

- Multi-File Projects
- `project.json` Manifest
- `.layout.json` Schema

Frontend Integration Guide

- What Reid Needs to Build
- Emitter Requirements Summary

Compiler

- Design Decisions

- Compiler API
- Multi-File Compilation
- Auto-Index Resolution
- DRC Engine
- Layout Validation
- Deterministic ID Generation
- WASM Exports
- Python Exports
- What We Are NOT Building

Backend Data Model

- Database Schema
- API Endpoints
- Migration Strategy

Language Reference

- Lexical Structure
- Grammar
- Statements
- Common Productions
- Design Rule Checks (DRC)

Appendix — Design Decision Records

- Cards: Template with Meta vs Keyword
- Ring Member Syntax
- Slot/Card Compatibility
- ID Separator
- Reserved Keywords
- Project Structure: Page Tree vs Flat Bundle
- `project.json` : Full Manifest vs Inferred
- Multi-File Compilation: File Map
- Flat Namespace
- Snapshot Strategy